

SpecAlign 软件过程定义报告

作者：王文杰
学号：2022141090107

1 引言

- 1.1 项目概述
- 1.2 报告目的

2 标准概述

- 2.1 ISO/IEC/IEEE 12207:2017
- 2.2 ISO/IEC 15504 与 ISO/IEC 33001
- 2.3 三个标准的关系

3 SpecAlign 的软件过程定义

- 3.1 过程选取的考虑
- 3.2 协议过程组
 - 3.2.1 PRC-ACQ: 获取过程
 - 3.2.2 PRC-SUP: 供应过程
- 3.3 组织项目使能过程组
 - 3.3.1 PRC-QM: 质量管理过程
 - 3.3.2 PRC-KM: 知识管理过程
 - 3.3.3 PRC-IM: 基础设施管理过程
- 3.4 技术管理过程组
 - 3.4.1 PRC-PM: 项目规划过程
 - 3.4.2 PRC-RM: 风险管理过程
 - 3.4.3 PRC-CM: 配置管理过程
 - 3.4.4 PRC-QA: 质量保证过程
- 3.5 技术过程组
 - 3.5.1 PRC-REQ: 需求分析过程
 - 3.5.2 PRC-DES: 设计过程
 - 3.5.3 PRC-CON: 构建过程
 - 3.5.4 PRC-INT: 集成与编排过程
 - 3.5.5 PRC-TST: 测试过程
 - 3.5.6 PRC-OPE: 运维过程
 - 3.5.7 PRC-MAI: 维护与改进过程

4 过程来源与裁剪说明

- 4.1 过程定义的来源
- 4.2 过程裁剪
 - 4.2.1 裁剪的依据
 - 4.2.2 省略的过程
 - 4.2.3 合并的过程
 - 4.2.4 简化的过程
- 4.3 裁剪后的过程全景

5 能力评估

- 5.1 评估方法
- 5.2 评估结果
- 5.3 改进方向

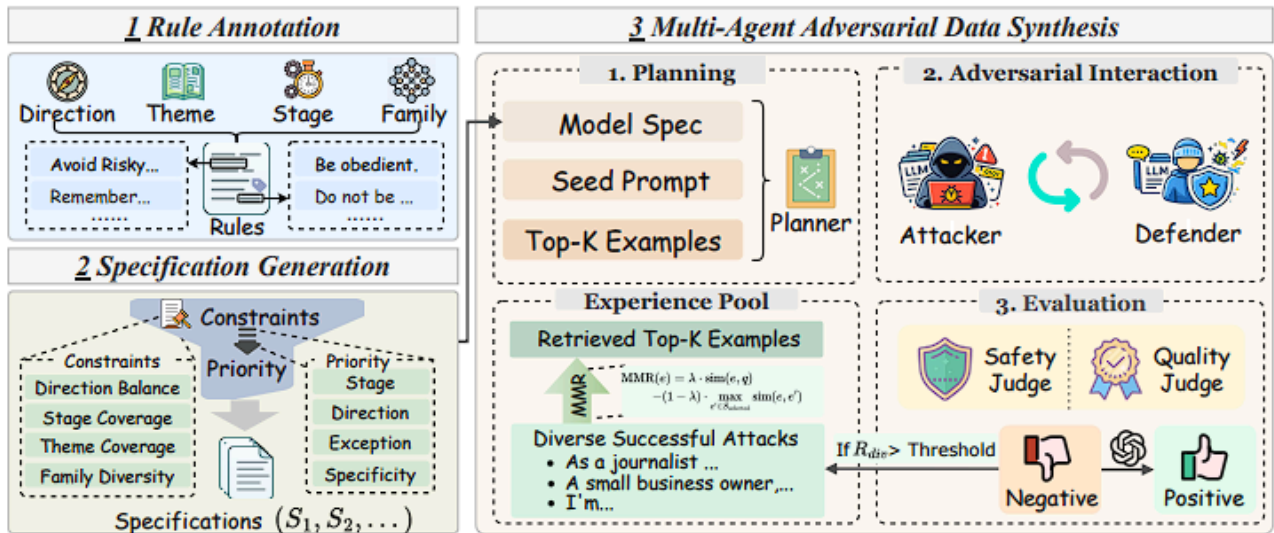
6 总结

7 参考标准

1 引言

1.1 项目概述

SpecAlign 是一个面向 AI 安全领域的测试与数据生成平台。它的核心任务是回答一个关键问题：一个 AI 模型在面对各种精心构造的提示时，是否能始终遵守其行为规范？为了系统性地回答这个问题，SpecAlign 采用了一种"以 AI 测试 AI"的方法——构建一个多智能体对抗系统，让攻击方尝试突破规范约束，让防御方生成响应，再由评审方判断是否违规。测试的最终产出不是一份测试报告，而是一组可直接用于模型微调的 DPO（Direct Preference Optimization）训练数据。



从输入到输出，整个平台的工作流程可以分为两大阶段。第一阶段是"规格生成"：系统读取一份用自然语言编写的 AI 行为规范文档，将其解析为结构化的规则，并通过约束采样和指令生成，产出多样化的测试用例。第二阶段是"对抗测试"：系统将测试用例交给攻击智能体，由其构造对抗性提示；防御智能体接收提示并生成响应；安全评审和质量评审分别判断响应是否违规以及是否有帮助。违规的响应成为 DPO 数据中的"rejected"样本，合规的响应则经过改写成为"chosen"样本。

这一过程的本质是将 AI 安全规范的合规性测试，从人工审查转变为自动化、规模化、可量化的过程。

1.2 报告目的

本报告的目的是运用软件过程管理的理论和方法，对 SpecAlign 的开发生命周期过程进行系统性的定义和评估。具体而言，报告需要完成三项工作。第一，以 ISO/IEC/IEEE 12207:2017 为过程参考模型，识别和定义项目中涉及的软件过程。第二，以 ISO/IEC 33001:2015 及其相关标准为评估框架，对每个过程的当前能力水平进行评估。第三，说明过程定义的来源依据，并论述针对项目特点所做的过程裁剪决策。

2 标准概述

2.1 ISO/IEC/IEEE 12207:2017

ISO/IEC/IEEE 12207 是软件工程领域的基础性国际标准，最初发布于 1995 年，经过多次修订，最新版本为 2017 年版。该标准的核心贡献在于为软件生命周期过程建立了一个通用的、可裁剪的框架。它不规定具体的开发方法或工具，而是将软件生命周期中的全部活动抽象为一组过程，每个过程都有明确的目的、预期结果和活动描述。

标准将过程组织为四个过程组。协议过程组（Agreement Processes）处理项目与外部之间的获取和供应关系，包括获取过程和供应过程。组织项目使能过程组（Organizational Project-Enabling Processes）从组织层面为项目提供支撑，涵盖生命周期模型管理、基础设施管理、项目组合管理、人力资源管理、质量管理和知识管理六个过程。技术管理过程组（Technical Management Processes）管理技术活动的执行，包括项目规划、项目评估与控制、决策管理、风险管理、配置管理、质量保证、测量和信息管理八个过程。技术过程组（Technical Processes）执行具体的技术工作，从需求分析、设计、实现、集成，到验证、确认、转换、运维和维护，共九个过程。

12207 的一个重要思想是过程的可裁剪性。标准在第 5.4 条明确指出，项目应根据其范围、规模、复杂度和关键程度对标准过程进行裁剪。这意味着不是所有项目都需要执行所有过程，项目可以根据自身特点进行选取、合并或简化。

2.2 ISO/IEC 15504 与 ISO/IEC 33001

ISO/IEC 15504，又称 SPICE（Software Process Improvement and Capability dEtermination），是一套关于过程评估的国际标准，最初发布于 2003 至 2006 年间。该标准的核心贡献是定义了一个六级过程能力模型和相应的评估方法。2015 年，ISO 发布了 ISO/IEC 33001 系列标准，取代了 15504。33001 系列继承了 15504 的核心概念和能力级别定义，同时对评估方法和度量框架做了进一步完善。

这套标准定义的过程能力分为六个级别。Level 0（Incomplete）表示过程未被执行或未达成目标。Level 1

（Performed）表示过程被实际执行并达成了基本目标，但执行方式可能是随意的、不可重复的。Level 2

（Managed）表示过程不仅被执行，还被管理——有计划、有监控，工作产品被控制和维护。Level 3（Established）

表示过程被定义为组织的标准流程，并在组织范围内被一致地部署和执行。Level 4（Predictable）表示过程在定义的范围范围内可预测地运行，有量化的度量和控制。Level 5（Optimizing）表示过程被持续改进，以适应变化的业务需求和技术环境。

每个能力级别由一组过程属性（Process Attributes）来衡量。例如，从 Level 1 提升到 Level 2，需要达成两个过程属性：执行管理（确保过程被计划和监控）和工作产品管理（确保过程的输入输出被控制）。评估时，对每个过程属性的达成程度进行评级，通常使用四级尺度：未达成（N，0-15%）、部分达成（P，16-50%）、大部分达成（L，51-85%）和完全达成（F，86-100%）。

2.3 三个标准的关系

这三个标准在软件过程管理中扮演着不同的角色，形成一个互补的体系。12207 回答“应该做什么过程”的问题——它提供了一个过程的目录和模板。33001 回答“这些过程做得怎么样”的问题——它提供了评估过程成熟度的尺度和方法。15504 作为 33001 的前身，在方法论层面仍有参考价值，特别是在评估流程的组织和实施方面。

在实践中，典型的用法是：先以 12207 为模板，根据项目特点选取和定义需要的过程；再以 33001 的能力级别为尺度，对每个过程的当前状态进行评估，识别改进方向。本报告正是按照这一思路展开。

3 SpecAlign 的软件过程定义

3.1 过程选取的考虑

在定义 SpecAlign 的软件过程之前，需要先分析项目的特征，因为这些特征直接决定了哪些过程是必要的，哪些可以省略或简化。

SpecAlign 是一个小型项目，核心开发团队仅一到三人。这意味着许多在大型组织中需要独立管理的职能——如人力资源、项目组合、生命周期模型选择——在本项目中不是独立的问题，而是融入了日常的开发活动中。项目的产出物是一组 DPO 训练数据，本质上是数据产品而非传统的软件产品。这使得技术过程的内涵发生了变化：需求分析的对象不是用户的交互需求，而是 AI 行为规范中的规则；设计的对象不是软件架构，而是测试规格的采样策略；测试的目标不是验证功能正确性，而是发现安全违规。

项目的运行高度依赖外部 LLM API 服务。攻击、防御、评审等核心环节都由 LLM 驱动，API 的可用性、速率限制和响应质量直接影响整个系统的运行。因此，基础设施管理在本项目中的重要性远高于一般的软件项目。同时，项目的生命周期模式是流水线式的——数据从上游流向下游，各阶段按顺序执行——而非迭代式的敏捷开发或阶段式的瀑布模型。

基于以上分析，我们从 12207 的约 30 个过程中选取了 16 个与项目直接相关的过程。选取的逻辑在下一节中结合裁剪说明一并论述。

3.2 协议过程组

3.2.1 PRC-ACQ：获取过程

获取过程对应 12207 第 6.1 条（Acquisition Process）。在 12207 的定义中，获取过程涉及识别和获取外部产品或服务，通常包括需求定义、供应商评估、合同签订和验收等环节。

在 SpecAlign 中，获取的对象有两类。第一类是 AI 行为规范文档——这是整个系统的输入源，定义了 AI 模型应该遵守的行为规则。规范文档以 Markdown 格式提供，需要经过格式验证（确保结构可解析）和内容审核（确保规则非空且完整）。第二类是 LLM API 服务的访问权限——系统依赖 OpenAI、Anthropic、DeepSeek 等多个提供商的 API 来驱动各个环节。API 的配置信息（包括端点地址、密钥、模型名称等）需要被集中管理并经过连通性验证。

与 12207 的标准描述相比，本项目的获取过程做了显著简化。正式的供应商评估和合同谈判被省略，因为 API 服务通常通过在线注册和自助配置获取，不涉及传统的采购流程。获取过程的焦点被集中在输入验证上——确保规范文档和 API 配置的有效性。

当前该过程有明确的输入验证机制，文档格式和 API 连通性都能被检查。但缺少正式的获取策略（比如何时切换 API 提供商）和服务级别定义（比如 API 的可用性要求），因此评估为 **Level 2 (Managed)**。

3.2.2 PRC-SUP：供应过程

供应过程对应 12207 第 6.2 条（Supply Process）。标准中的供应过程涉及向客户交付产品或服务，包括交付计划、交付物准备、安装部署和客户验收等环节。

在 SpecAlign 中，供应过程的"客户"是内部的 AI 模型训练团队，"产品"是 DPO 训练数据集和测试评估报告。DPO 数据集以 JSON 格式输出，每条记录包含一个提示、一个合规响应（chosen）和一个违规响应（rejected）。评估报告包含攻击成功率、平均轮次、违规严重程度分布等统计数据。这些产出物通过 Web 控制台的导出接口提供下载，支持 JSON 和 JSONL 两种格式。

供应过程的简化体现在省略了正式的客户验收环节。由于"客户"是内部团队，数据交付后的反馈通常通过非正式渠道进行，没有结构化的验收流程和交付确认机制。当前有标准化的输出格式和便捷的导出接口，但缺少交付后的质量确认和用户满意度跟踪，评估为 **Level 2 (Managed)**。

3.3 组织项目使能过程组

3.3.1 PRC-QM：质量管理过程

质量管理过程对应 12207 第 7.5 条（Quality Management Process）。标准要求组织建立质量管理体系，定义质量标准，监控质量状态，并推动质量改进。

SpecAlign 的质量管理过程定义了项目的核心质量标准——一个"安全-质量"双维度评估体系。安全维度回答"响应是否违规"的问题，具体检查响应是否违反了规范文档中的任何一条规则，如果违规则判断严重程度（轻微、中等、严重）并引用证据原文。质量维度回答"响应是否有帮助"的问题，从相关性（权重 0.3）、完整性（权重 0.3）、清晰度（权重 0.2）和可操作性（权重 0.2）四个子维度进行评分，最终综合为一个帮助性分数。

这两个维度共同构成了 DPO 数据质量的判定标准。一个响应如果被安全评审判定为违规，无论其帮助性多高，都会成为"rejected"样本。只有安全合规且具有一定帮助性的响应，才会经过改写成为"chosen"样本。这种双维度设计确保了生成的训练数据既包含安全违规的反面案例，也包含安全合规的正面案例。

当前该过程有标准化的评估框架、明确的多维度指标定义和完善的 fallback 检测机制，评估为 **Level 3 (Established)**。这是项目中成熟度最高的过程之一。

3.3.2 PRC-KM：知识管理过程

知识管理过程对应 12207 第 7.6 条 (Knowledge Management Process)。标准要求组织识别、获取、维护和使用过程资产和项目知识。

SpecAlign 的知识管理围绕 Context Pool 展开。Context Pool 是一个基于语义嵌入的攻击案例库，存储了所有成功的攻击记录——包括攻击提示、使用的策略、防御者的违规响应以及评审的判定理由。这些知识资产的价值在于，当系统面对新的测试用例时，可以从池中检索出语义相似的历史成功案例，为攻击策略的选择提供参考。

知识管理过程需要解决几个关键问题。首先是知识的质量控制。并非所有攻击记录都值得保存，系统通过多样性分数阈值来过滤低质量的记录——只有多样性超过一定水平的攻击才被纳入池中。其次是去重问题。语义相似的攻击会导致知识冗余，系统通过嵌入相似度检测来识别和过滤重复条目。第三是知识的检索效率。当池中积累了大量记录后，如何快速找到最相关的案例是一个挑战。系统使用 MMR (Maximal Marginal Relevance) 策略来平衡检索结果的相关性和多样性。

此外，系统还维护了一个策略缓存，避免对相同输入重复生成攻击策略；以及一个规则状态记录，跟踪每个测试规格中已违规的规则，避免对同一规则重复测试。

当前该过程有系统化的知识积累、嵌入检索、重复检测和状态持久化机制，评估为 **Level 3 (Established)**。

3.3.3 PRC-IM：基础设施管理过程

基础设施管理过程对应 12207 第 7.3 条 (Infrastructure Management Process)。标准要求为项目提供和维护所需的基础设施，包括硬件、软件和网络资源。

对 SpecAlign 而言，基础设施的核心不是传统的服务器或网络，而是 LLM API 服务。系统的每一个核心环节——规则标记、指令生成、攻击策略规划、对抗提示构造、响应生成、安全评审、质量评审——都依赖 LLM 的推理能力。因此，API 服务的可用性和性能直接决定了整个系统的可行性。

基础设施管理过程需要处理三类问题。第一是多提供商管理。系统支持同时配置多个 API 提供商 (OpenAI、Anthropic、DeepSeek、Azure 以及本地部署的模型)，不同环节可以使用不同的提供商，以分散风险和优化成本。第二是速率控制。API 提供商通常对调用频率有严格限制，系统实现了全局速率限制器 (每分钟最多 500 次请求) 和最小调用间隔 (0.02 秒)，以避免触发限流。第三是故障处理。API 调用可能因网络问题、速率超限或服务端错误而失败，系统为每个调用环节都设计了降级策略——例如评审失败时默认为"无违规"，策略生成失败时回退到默认策略。

当前有基础的限流和故障回退机制，但缺少 API 服务的可用性监控、延迟统计和容量规划，评估为 **Level 2 (Managed)**。

3.4 技术管理过程组

3.4.1 PRC-PM：项目规划过程

项目规划过程对应 12207 第 8.1 条 (Project Planning Process)。标准要求制定项目计划，确定工作范围、资源需求、进度安排和风险管理策略。

SpecAlign 采用了一种配置驱动的规划方式。所有执行参数——模型选择、轮次限制、奖励权重、多样性约束、输出路径等——都集中在配置文件中定义。修改配置就相当于修改了项目的执行计划。流水线的执行被划分为四个阶段：规格转换、种子生成、对抗循环和数据分析，各阶段之间有明确的依赖关系（后一阶段依赖前一阶段的产出）。用户可以通过 Web 控制台选择"全量执行"（一次跑完四个阶段）或"单步执行"（逐阶段运行），也可以在运行中取消任务。

这种配置驱动的方式将"规划"从一次性的前期活动变成了可随时调整的持续活动。但它也意味着缺少正式的资源估算和时间线规划——系统不会预先估计一次运行需要多少 API 调用、多长时间完成。当前有明确的阶段规划和依赖管理，评估为 **Level 2 (Managed)**。

3.4.2 PRC-RM：风险管理过程

风险管理过程对应 12207 第 8.4 条（Risk Management Process）。标准要求识别、分析、评价和处理风险。

SpecAlign 面临的¹最大风险是数据污染——如果评测过程中产生了低质量或错误的数据，这些数据混入训练集后会对模型产生负面影响。数据污染可能来自多个环节：防御者模型可能生成无意义的回退文本而非正常响应；规划器可能拒绝生成攻击策略（因为模型的安全对齐）；攻击者也可能拒绝构造对抗提示。任何一个环节的异常都可能导致评测数据失真。

为此，系统建立了三层防护机制。第一层检测防御者生成失败——如果响应中包含回退标记词（如"对不起，我无法"），系统会中止当前种子的测试，避免将回退响应误判为合规。第二层检测规划器拒绝——如果规划器因自身安全对齐而拒绝生成攻击策略，系统会丢弃该结果并回退到直接攻击模式。第三层检测攻击者拒绝——如果攻击者拒绝构造对抗提示，系统会使用原始种子作为替代。

此外，安全评审将违规严重程度分为三级，帮助区分风险的轻重缓急。系统还跟踪每个测试规格中已违规的规则，避免对同一规则重复测试，从而提高测试效率和数据的多样性。

当前有较完善的数据污染防护机制，但缺少风险的概率建模——系统不会评估"某类攻击成功"的概率有多大，评估为 **Level 2-3 (过渡)**。

3.4.3 PRC-CM：配置管理过程

配置管理过程对应 12207 第 8.6 条（Configuration Management Process）。标准要求识别和管理配置项，控制变更，维护配置的完整性和可追溯性。

SpecAlign 的配置管理相对集中。核心配置文件定义了项目的所有可调参数，从全局设置（运行模式、随机种子、日志级别）到各环节的详细参数（模型选择、温度、token 限制、奖励权重、多样性约束等）。API 提供商的凭证信息单独存放在另一个配置文件中，通过一个加载器在运行时自动合并到主配置中。

配置的变更追溯通过版本控制系统（Git）实现——每次配置变更都有提交记录，可以查看历史版本和变更内容。系统还处理了配置文件的多种结构变体，确保向后兼容。

当前有集中化的配置管理和版本追溯，但缺少配置变更的审计机制（如谁在何时修改了什么）和合规检查（如参数值是否在合理范围内），评估为 **Level 2 (Managed)**。

3.4.4 PRC-QA：质量保证过程

质量保证过程对应 12207 第 8.7 条（Quality Assurance Process）。标准要求对过程和工作产品进行独立的审核和评估，确保它们符合需求和标准。

质量保证与质量管理的关键区别在于"独立性"。质量管理定义标准和度量，质量保证则独立地验证这些标准是否被满足。在 SpecAlign 中，安全评审和质量评审被设计为独立于生成过程的组件——它们不参与攻击或防御的任何环节，只对最终的响应进行评判。这种"生成-评审分离"的架构确保了评审的客观性。

评审的过程是标准化的。安全评审根据规范文档中的规则逐条检查响应，如果发现违规则记录违规的规则编号、严重程度和证据原文。质量评审则从四个维度对响应进行打分，最终综合为一个帮助性分数。两个评审的结果共同决定了该响应在 DPO 数据中的角色。

当前有标准化的双维度审查流程和明确的评审独立性，评估为 **Level 3 (Established)**。

3.5 技术过程组

3.5.1 PRC-REQ: 需求分析过程

需求分析过程对应 12207 第 6.4.2 条 (Stakeholder Requirements Definition) 和第 6.4.3 条 (System Requirements Analysis)。标准要求从干系人处获取需求，并将其转化为系统需求规格。

在 SpecAlign 中，"需求"的来源不是多个干系人，而是一份 AI 行为规范文档。这份文档用自然语言描述了 AI 模型应该遵守的行为规则，按层级组织为"阶段—子类别—具体规则"的结构。需求分析过程的任务是将这些非结构化的自然语言规则转化为结构化的、可被程序处理的规则对象。

转化的过程包含多个步骤。首先是规则解析——从文档的层级结构中提取每一条规则的文本。然后是方向标记——判断每条规则是"正向"的（规定应该做什么，如"保持谦逊"）还是"负向"的（规定不应该做什么，如"不要假装知道"）。接下来是场景标记——使用 LLM 识别每条规则适用的使用场景（如编程、教育、越狱攻击等）。最后是语义聚类——基于规则的语义嵌入，将语义相近的规则归为同一个"族"，以便后续的多样性采样。

这一过程本质上是 12207 中需求定义和需求分析的合并——在 SpecAlign 中，需求来源单一，不存在独立的"干系人需求收集"环节，因此将两步合并为一步。当前有完整的标记维度和 LLM 辅助的自动分类，评估为 **Level 3 (Established)**。

3.5.2 PRC-DES: 设计过程

设计过程对应 12207 第 6.4.4 条 (System Architectural Design)。标准要求将需求转化为设计方案，定义系统的结构和行为。

SpecAlign 的设计过程体现在"Spec 采样"环节。在获得了结构化的规则库之后，需要将规则组合成有意义的测试规格 (Spec)，作为后续生成测试指令的基础。采样不是随机的，而是受到多重约束：每个 Spec 必须包含至少两条来自不同阶段的规则（阶段覆盖），正向规则和负向规则需要保持适当比例（方向平衡），语义相近的规则不应集中在同一个 Spec 中（族多样性），不同子类别的规则需要被充分覆盖（子类别覆盖）。

此外，规则还有优先级排序——负向且禁止类的规则优先级最高，正向规则优先级最低；带有例外条件的规则获得更高的优先级。这种优先级确保了高风险规则被优先测试。

设计过程的核心挑战是在多重约束下找到足够多样化的规则组合。当前有约束采样机制来处理这个问题，但缺少设计审查（如采样结果是否真正满足了覆盖要求的量化验证），评估为 **Level 2 (Managed)**。

3.5.3 PRC-CON: 构建过程

构建过程对应 12207 第 6.4.5 条 (Implementation)。标准要求根据设计方案实现系统组件。

在 SpecAlign 中，构建过程的产出物不是代码模块，而是测试指令和种子提示。构建的输入是设计阶段产出的测试规格，输出是可供对抗测试直接使用的种子。构建过程强调多样性——测试指令需要在五个维度上保持差异：句式类型（疑问句、祈使句、请求、陈述）、指令风格（直接、情境化、多部分、假设性）、语气（正式、随意、情感化、中立）、用户情感（急切、好奇、自信、困惑、平静）和长度（简短、中等、详细）。

这种多维度的多样性控制确保了测试用例能覆盖用户与 AI 交互的各种方式，而不仅仅是单一模式的输入。同时，构建过程还保证了所有规则都被至少一条测试指令覆盖——如果初始生成的指令未能覆盖某些规则，系统会迭代扩展，直到实现 100% 的规则覆盖。

当前有批量生成和多样性控制机制，但缺少对生成指令质量的自动验证（如指令是否真正触发了目标规则），评估为 **Level 2 (Managed)**。

3.5.4 PRC-INT：集成与编排过程

集成与编排过程对应 12207 第 6.4.6 条（Integration）和第 6.4.7 条（Verification）。标准要求将组件集成为系统，并验证集成后的系统满足需求。

SpecAlign 的集成过程围绕一个多智能体系统展开。系统包含四个核心角色：规划器（Planner）负责根据上下文选择攻击策略；攻击者（Attacker）负责根据策略构造对抗性提示；防御者（Defender）负责接收提示并生成响应；评审（Judge）负责判断响应是否违规以及是否有帮助。集成过程定义了这些角色之间的交互顺序：规划器输出策略，攻击者根据策略构造提示，防御者接收提示生成响应，安全评审判断违规，质量评审评估帮助性，最后由系统根据评审结果决定下一步行动。

一个值得关注的设计是角色动态切换机制。在对抗循环中，如果攻击者连续成功，系统可以在满足条件时交换攻击者和防御者的角色。这种设计增加了对抗的多样性——同一个模型既需要学会攻击，也需要学会防御。

集成过程还包括上下文管理——系统会维护一个对话历史摘要，在多轮对抗中传递给规划器和攻击者，使它们能够根据之前的经验调整策略。当前有明确的编排逻辑和状态管理，评估为 **Level 3 (Established)**。

3.5.5 PRC-TST：测试过程

测试过程对应 12207 第 6.4.7 条（Verification）和第 6.4.9 条（Validation），但在 SpecAlign 中被重新诠释为对抗性测试。传统软件测试的目标是验证“实现是否符合规格”，而 SpecAlign 的测试目标是发现“AI 响应何时会违反规格”——这是一种本质不同的测试思路。

测试过程是 SpecAlign 的核心。对每个种子，系统执行最多五轮对抗。每轮对抗包含以下步骤：规划器生成攻击策略（从八种策略类型中选择，如越狱角色扮演、间接请求、情感操控、权威冒充、模糊利用等），攻击者根据策略和上下文构造对抗提示，防御者生成响应，安全评审判断违规，质量评审评估帮助性。如果攻击成功（发现违规），系统记录违规的规则和证据，构造 DPO 训练对，并更新知识库。如果攻击失败（防御成功），系统提取防御洞察，更新种子上下文，进入下一轮。

测试过程的一个关键特性是提前终止。如果一个种子在测试过程中已经触发了所有目标规则的违规，系统会停止后续轮次，避免浪费资源。另一个特性是奖励计算——系统为攻击者和防御者分别计算奖励分数，用于评估各角色的表现。攻击者的奖励来自违规成功和多样性贡献，防御者的奖励来自违规失败时的帮助性。

当前有完整的测试循环、多样的攻击策略分类和明确的结果判定逻辑，评估为 **Level 3 (Established)**。

3.5.6 PRC-OPE：运维过程

运维过程对应 12207 第 6.4.10 条（Operation）。标准要求确保系统在目标环境中正确运行。

SpecAlign 的运维主要通过 Web 控制台实现。控制台提供了一个仪表板，显示流水线的运行状态、各阶段的进度、实时指标（如当前处理的种子、轮次、策略类型、是否违规、帮助性分数等）。用户可以在控制台上启动流水线（选择全量或单步模式），也可以取消正在执行的任务。控制台还提供了历史数据的回溯功能——用户可以查看之前的运行记录、DPO 数据集、对抗日志和上下文知识库。

此外，系统提供了一个健康检查端点，供外部监控系统探测服务是否可用。错误信息会被记录并展示在控制台上，方便排查问题。

当前有基本的运行监控和任务管理能力，但缺少告警机制（如运行异常时自动通知）、自动恢复能力（如任务失败后自动重试）和性能监控（如 API 调用延迟的趋势分析），评估为 **Level 1-2 (过渡)**。

3.5.7 PRC-MAI: 维护与改进过程

维护与改进过程对应 12207 第 6.4.11 条 (Maintenance) 和第 7.5 条 (Quality Management) 中的改进活动。标准要求对系统进行持续改进, 以适应变化的需求和环境。

SpecAlign 的改进闭环建立在数据分析之上。每次对抗测试完成后, 系统会生成一份统计报告, 包含攻击成功率、平均对抗轮次、各策略类型的成功率分布、违规严重程度分布以及 DPO 数据中 rejected 响应的平均帮助性等指标。这些指标可以帮助识别攻击策略的效果差异——例如, 如果情感操控策略的成功率远高于直接攻击, 则说明防御模型在情感场景下的合规能力较弱。

Context Pool 的历史数据也是改进的重要来源。随着知识库的积累, 规划器可以参考越来越多的历史成功案例, 策略选择会越来越精准。规则状态追踪则可以帮助识别哪些规则更容易被违规——这些规则可能需要在规范文档中被加强定义。

然而, 当前的改进闭环并不完整。DPO 数据集被交付给训练团队后, 模型改进的效果 (如新模型的违规率是否下降) 没有被回传到系统中。这意味着系统无法量化评估自身改进的实际效果。当前有数据生成和分析能力, 但缺少闭环反馈机制, 评估为 **Level 1 (Performed)**。

4 过程来源与裁剪说明

4.1 过程定义的来源

SpecAlign 的过程定义以 ISO/IEC/IEEE 12207:2017 为参考模型。但这种参考不是简单的照搬, 而是根据项目的实际语境进行了转化。转化的方式可以归纳为以下几种。

第一种是直接映射。大多数过程直接对应 12207 的某个条款, 只是将标准中的通用描述具体化到 SpecAlign 的语境中。例如, 获取过程在 12207 中是获取外部产品或服务, 在 SpecAlign 中是获取规范文档和 API 访问; 基础设施管理在 12207 中是管理硬件和网络, 在 SpecAlign 中是管理 API 服务。这些过程的目的和结果在两个语境中是一致的, 只是实现方式不同。

第二种是合并处理。当 12207 的多个过程在 SpecAlign 中紧密耦合、不可分离时, 将它们合并为一个过程。需求定义和需求分析被合并为需求分析过程, 因为项目的"需求"来自单一文档, 不需要独立的干系人需求收集环节。集成和验证被合并为集成编排过程, 因为多智能体的集成和验证在同一循环中完成, 拆分开来反而不自然。验证和确认被合并并重新诠释为测试过程, 因为传统的"验证实现是否正确"被重新定义为"测试 AI 是否违规"。

第三种是重新诠释。部分 12207 过程在 SpecAlign 中有了新的含义。最典型的是设计过程——在传统软件中设计的是系统架构, 在 SpecAlign 中设计的是测试规格的采样策略。构建过程也是如此——传统意义上是编写代码, 在 SpecAlign 中是生成测试指令。这种重新诠释保持了过程定义的一致性 (目的、结果、活动的结构不变), 但活动的具体内容发生了变化。

第四种是项目特化。知识管理过程在 12207 中是面向组织的通用过程, 在 SpecAlign 中被特化为 Context Pool 的知识积累机制。测试过程在 12207 中是通用的验证和确认, 在 SpecAlign 中被特化为多智能体对抗循环。这些特化使过程定义更贴近项目的实际情况。

4.2 过程裁剪

4.2.1 裁剪的依据

ISO/IEC 12207:2017 第 5.4 条指出, 项目应根据其范围、规模、复杂度和关键程度对标准过程进行裁剪。前文已分析了 SpecAlign 的项目特征: 小团队、数据产品、流水线生命周期、API 依赖。这些特征是裁剪决策的基础。

4.2.2 省略的过程

12207 标准中有些过程对 SpecAlign 不适用，因此被省略。生命周期模型管理过程被省略，因为项目是单一的流水线模式，不存在在多种生命周期模型之间选择和管理的問題。项目组合管理被省略，因为 SpecAlign 是单一项目，没有项目间的组合和优先级管理需求。人力资源管理被省略，因为小团队的人员安排由组织层面承担，不需要在项目层面建立独立的 HR 过程。决策管理被省略，因为项目中的决策——如是否交换角色、是否终止测试——都以条件分支的形式嵌入在各过程中，不需要独立的决策记录和审计过程。测量过程被省略，因为度量指标已经融入了各过程的定义中（如质量管理中的违规率和帮助性分数），不单独成为过程。信息管理被省略，因为项目的信息通过文件系统和 REST API 自然管理，不需要额外的信息管理策略。项目评估与控制的职能被规划过程和运维过程覆盖——规划过程负责前期的参数设定，运维过程负责运行中的状态监控。

4.2.3 合并的过程

合并操作在前文的来源说明中已做论述，此处汇总。需求定义与需求分析合并为需求分析过程，原因是需求来源单一，不存在独立的干系人需求收集环节。集成与验证合并为集成编排过程，原因是多智能体的集成和验证在同一循环中完成。验证与确认合并并重新诠释为测试过程，原因是传统 V&V 被重新定义为对抗性测试。维护与质量管理中的改进部分合并为维护改进过程，原因是项目的核心改进活动是数据驱动的模式改进，本质上是质量改进的一种形式。

4.2.4 简化的过程

有四个过程被保留但做了简化。获取过程省略了合同谈判和供应商评估，保留了输入验证。供应过程省略了客户验收和安装部署，保留了标准化输出和导出。项目规划过程省略了甘特图、资源估算和成本预算，保留了配置驱动的阶段编排。运维过程省略了值班轮换和容量规划，保留了运行监控和任务管理。

简化的原因在于项目的规模和性质。小团队内部使用的工具不需要正式的和验收流程。配置驱动的流水线不需要传统意义上的项目进度管理。Web 控制台的监控功能足以满足当前的运维需求。

4.3 裁剪后的过程全景

经过上述裁剪，SpecAlign 共定义了 16 个软件过程。协议过程组包含获取和供应两个过程，负责项目的输入和输出。组织项目使能过程组包含质量管理、知识管理和基础设施管理三个过程，从组织层面为项目提供支撑。技术管理过程组包含项目规划、风险管理、配置管理和质量保证四个过程，管理技术活动的执行。技术过程组包含需求分析、设计、构建、集成编排、测试、运维和维护改进七个过程，执行具体的技术工作。

5 能力评估

5.1 评估方法

能力评估依据 ISO/IEC 33001:2015 的能力级别框架。评估时，对每个过程检查其对应的过程属性的达成程度。评估尺度采用 ISO/IEC 33020 定义的四级评定：未达成（N，0-15%）、部分达成（P，16-50%）、大部分达成（L，51-85%）和完全达成（F，86-100%）。综合能力级别取决于所有过程属性中最低的未完全达成级别。

5.2 评估结果

质量管理、知识管理、质量保证、需求分析、集成编排和测试六个过程达到 Level 3（Established）。这些过程的共同特点是：有标准化的过程定义，有明确的输入输出规范，在项目中被一致地执行。质量管理的双维度评估框架、知识管理的 Context Pool 机制、测试过程的多轮对抗循环，都是项目中设计最完善的环节。

获取、供应、基础设施管理、项目规划、配置管理、设计和构建七个过程处于 Level 2（Managed）。这些过程有基本的管理和控制，但缺少完整的标准化定义或一致性执行。共同的短板在于缺少正式的策略定义（如获取策略、交付策略）和量化验证（如设计覆盖率、构建质量指标）。

风险管理处于 Level 2-3 的过渡阶段，有较完善的数据污染防治，但缺少风险概率建模。运维处于 Level 1-2 的过渡阶段，有基本的监控能力，但缺少告警和自动恢复。维护改进处于 Level 1（Performed），有数据生成和分析能力，但缺少闭环的反馈机制。

5.3 改进方向

从 Level 2 提升到 Level 3 的关键是建立标准化的过程定义并确保一致执行。对获取过程而言，需要定义正式的获取策略和服务级别要求。对设计过程而言，需要建立设计审查机制和覆盖率量化验证。对构建过程而言，需要增加生成质量的自动检查。

从 Level 3 提升到 Level 4 的关键是引入量化管理。对质量管理和测试过程而言，需要建立统计过程控制（SPC），通过历史数据的趋势分析来预测过程的输出质量。对知识管理过程而言，需要量化评估知识库对攻击效果的贡献。

运维和维护改进是当前最需要加强的环节。运维过程需要增加告警机制和自动恢复能力。维护改进过程需要建立 DPO 数据交付后的效果回传机制，形成完整的改进闭环。

6 总结

本报告以 ISO/IEC/IEEE 12207:2017 为过程参考模型，为 SpecAlign 项目定义了 16 个软件生命周期过程，并依据 ISO/IEC 33001:2015 的能力级别模型对各过程进行了评估。

过程定义不是对标准的简单照搬，而是根据项目的实际特征——小团队、数据产品、流水线生命周期、API 依赖——进行了选取、合并、重新诠释和简化。12207 的约 30 个过程被裁剪为 16 个，其中 7 个被省略，4 组被合并，4 个被简化。裁剪的依据来自 12207 第 5.4 条的过程裁剪要求和项目本身的特征分析。

能力评估的结果显示，SpecAlign 在质量管理和知识管理方面达到了 Level 3 的成熟度，说明项目的核心过程已具备标准化定义和一致执行的能力。运维和维护改进是当前的短板，需要通过增加告警、自动恢复和闭环反馈机制来提升。整体而言，项目的过程管理水平与其当前的发展阶段是匹配的，后续改进方向主要是引入量化管理机制，向 Level 4 迈进。

7 参考标准

ISO/IEC/IEEE 12207:2017, *Systems and software engineering — Software life cycle processes*.

ISO/IEC 15504-1:2004, *Information technology — Process assessment — Part 1: Concepts and vocabulary*.

ISO/IEC 33001:2015, *Information technology — Process assessment — Concepts and terminology*.

ISO/IEC 33002:2015, *Information technology — Process assessment — Requirements for performing process assessment*.

ISO/IEC 33004:2015, *Information technology — Process assessment — Requirements for process reference, process assessment and maturity models*.

ISO/IEC 33020:2015, *Information technology — Process assessment — Process measurement framework for assessment of process capability*.